

P1 CAMINOS MÍNIMOS EN GRAFOS DE ATAQUE

Dijkstra encuentra la ruta de ataque

Modelar la red como grafo dirigido de CVEs reales. Peso $w = 10 - \text{CVSS}$: las vulnerabilidades críticas son el camino de menor resistencia.

ENTRADA

INTERNET

OBJETIVO

DB-CRITICAL

DATOS

CVEs reales · NVD

La red como grafo dirigido

◆ NODOS

Hosts y servicios

Cada activo de la infraestructura: firewall, servidor web, controlador de dominio, base de datos...

→ ARISTAS

CVEs explotables

Una arista $u \rightarrow v$ = "estando en u , explotar este CVE me lleva a v ".

$$W = 10 - CVSS$$

CVSS inverso. Vulnerabilidad crítica → peso bajo → **camino de menor resistencia** para el atacante.

CVSS 10 • w 0

CVSS 8 • w 2

CVSS 5 • w 5

CVSS 2 • w 8

★ ROL DE DIJKSTRA

Mínima resistencia

Camino de costo mínimo desde **INTERNET** hasta el **activo crítico** = la secuencia de ataque más probable.

CVEs reales del NVD

Vulnerabilidades reales con su **CVSS v3 oficial** de la National Vulnerability Database. API live con fallback a dataset embebido de CVEs conocidos.

CVE-2021-44228 Log4Shell — Apache Log4j 10.0 WEB	CVE-2017-0144 EternalBlue — SMBv1 8.1 SERVICE	CVE-2019-0708 BlueKeep — RDP 9.8 SERVICE	CVE-2020-1472 ZeroLogon — Netlogon 10.0 HOST
CVE-2019-19781 Citrix ADC / Gateway 9.8 PERIMETER	CVE-2021-26855 ProxyLogon — Exchange 9.8 SERVICE	CVE-2021-4034 PwnKit — Polkit pkexec 7.8 HOST	CVE-2018-13379 Fortinet FortiOS SSL VPN 9.8 PERIMETER

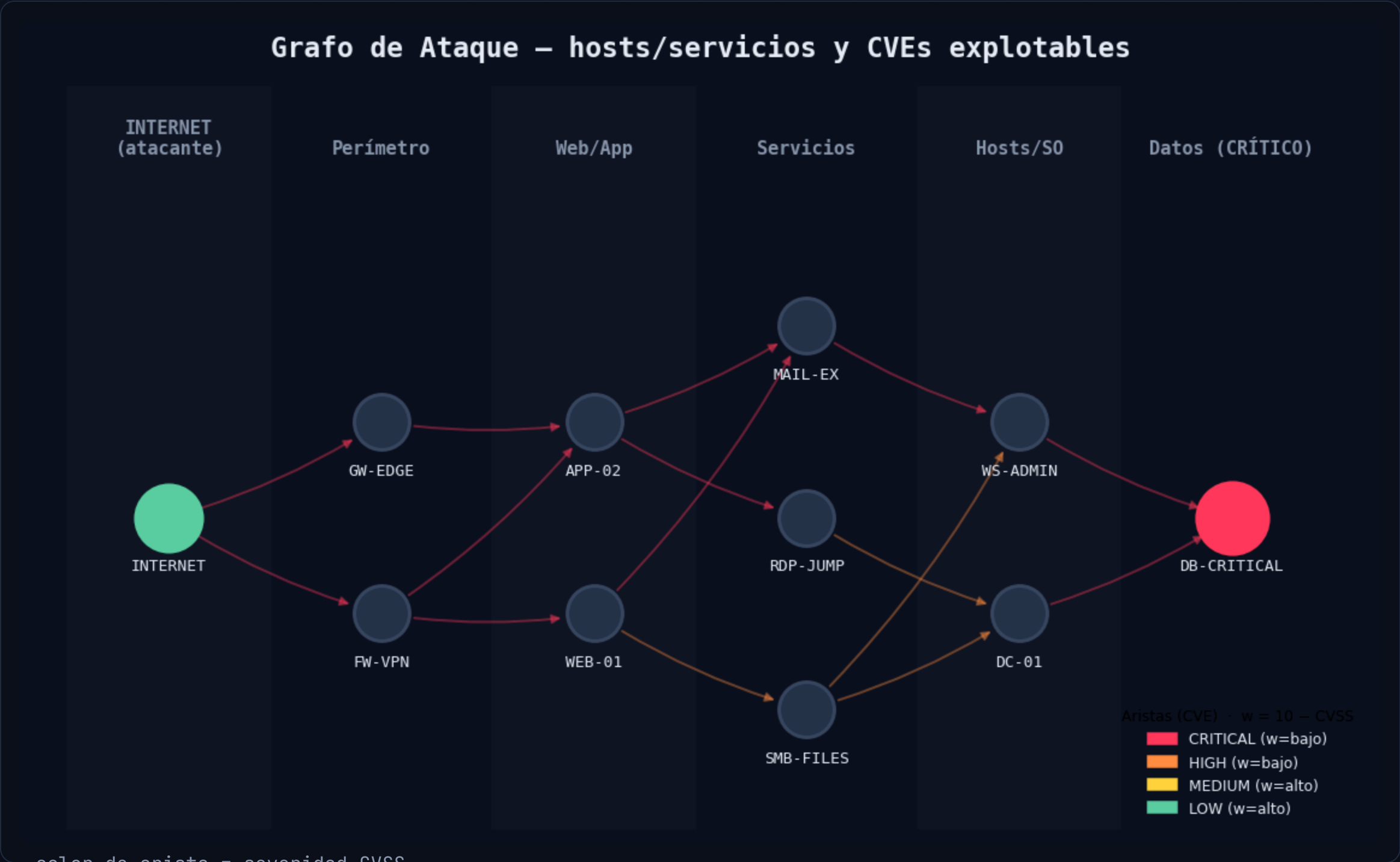
● 20 CVEs reales · clasificados por capa: perímetro → web → servicio → host → datos

De Internet al activo crítico

11 nodos

16 aristas (CVE)

6 capas



04 - CÓMO FUNCIONA

Dijkstra, paso a paso

01 Inicializar distancias

Todas en ∞ salvo el origen en 0. Min-heap con (distancia, nodo).

02 Extraer el más cercano

Sacar del heap el nodo de menor distancia acumulada y marcarlo como fijado.

03 Relajar vecinos

Para cada sucesor: si $\text{dist}[u] + w < \text{dist}[v]$, actualizar y registrar el predecesor.

04 Repetir hasta el objetivo

Al fijar **DB-CRITICAL**, reconstruir el camino por los predecesores.

```
// DIJKSTRA.PY - DESDE CERO
```

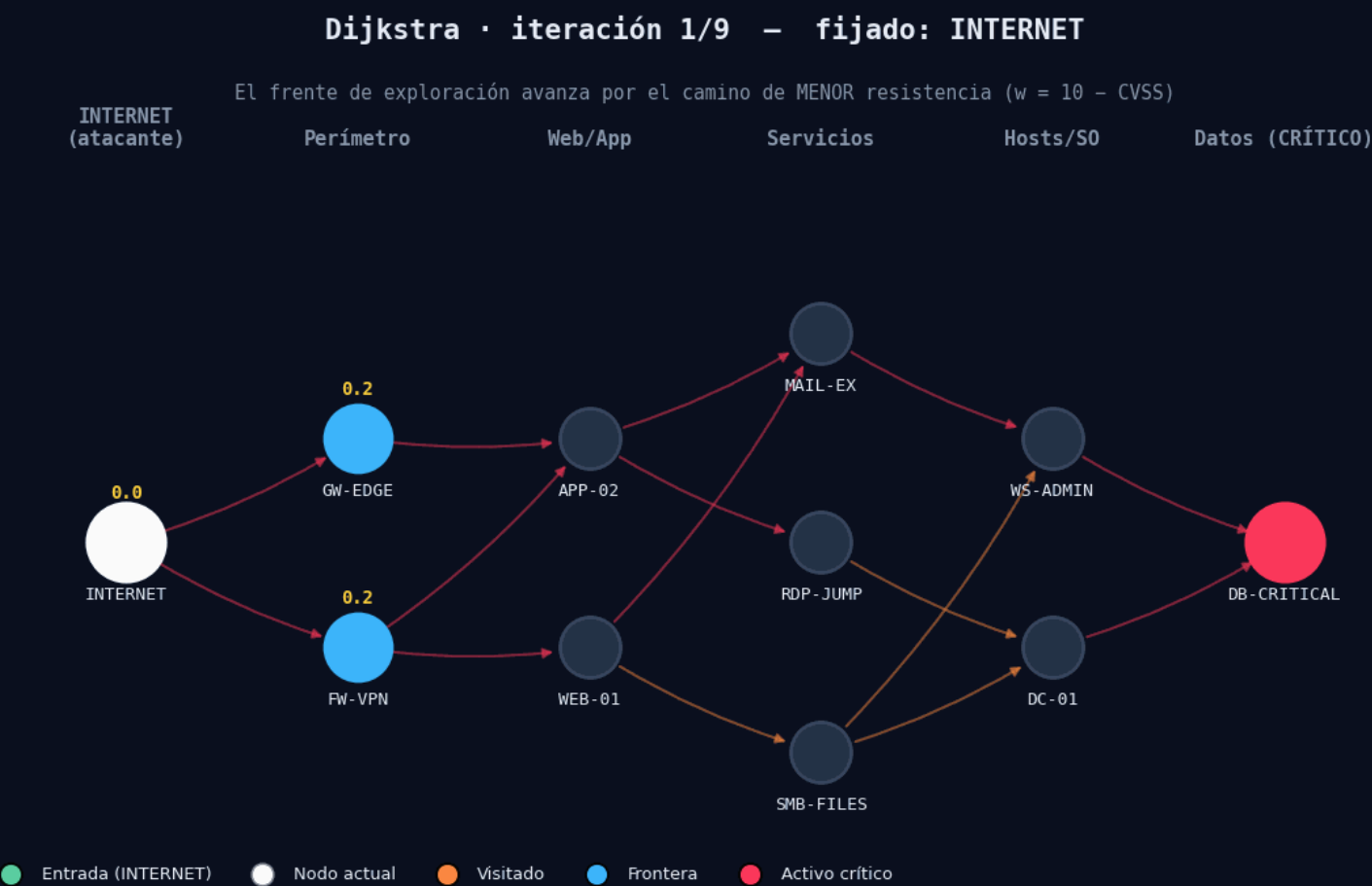
```
dist[s] = 0 # resto =  $\infty$   
pq = [(0, s)]
```

```
while pq:  
    d, u = heappop(pq)  
    if u in visited: continue  
    visited.add(u)
```

```
for v in succ(u):  
    w = 10 - CVSS  
    if d + w < dist[v]:  
        dist[v] = d + w  
        prev[v] = u  
        heappush(pq, (dist[v], v))
```


04 — EN ACCIÓN

El frente se expande



Cada iteración fija un nodo · números amarillos = resistencia mínima conocida

Dijkstra explora siempre el nodo de **menor distancia acumulada**. El frente avanza por las aristas más baratas (CVEs críticos).

- **Entrada** — INTERNET
- **Nodo actual** — recién fijado
- **Visitado** — distancia definitiva
- **Frontera** — en el heap
- **Activo crítico** — objetivo

9

ITERACIONES

$O(E \log V)$

COMPLEJIDAD

05 - RESULTADO

La **secuencia de ataque** más probable



La ruta se traza salto a salto • cada arista muestra el CVE explotado

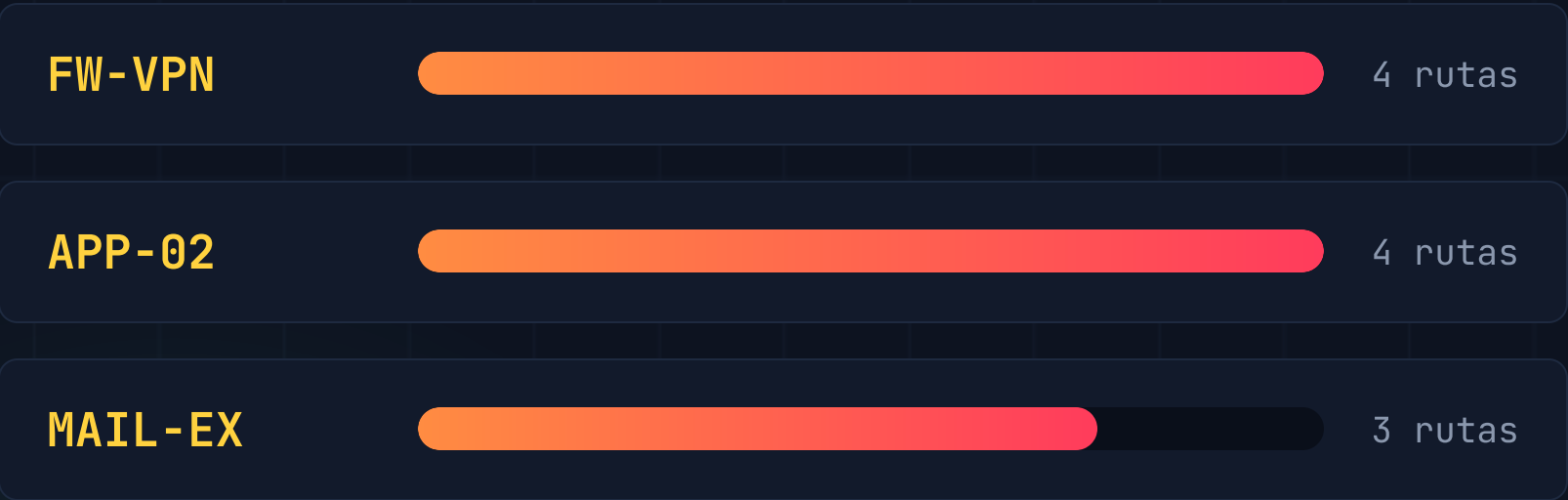
5 saltos desde Internet hasta la base de datos crítica — todos vía **CVEs CRITICAL** (CVSS ≥ 9.8), el camino de mínima resistencia.

1	INTERNET → FW-VPN	CVE-2023-27997	w 0.2
2	FW-VPN → APP-02	Log4Shell	w 0.1
3	APP-02 → MAIL-EX	BlueKeep	w 0.2
4	MAIL-EX → WS-ADMIN	BlueKeep	w 0.2
5	WS-ADMIN → DB-CRITICAL	CVE-2019-10149	w 0.2

RESISTENCIA TOTAL (Σ W) 0.90

Cuellos de botella = dónde defender

NODOS POR LOS QUE PASAN MÁS RUTAS DE BAJO COSTO



CORTAR UN CUELLO DE BOTELLA INVALIDA MÚLTIPLES RUTAS DE ATAQUE A LA VEZ.



Parchar la ruta crítica primero

Subir el CVSS→bajar el peso de los CVEs de la ruta **aumenta su resistencia** y fuerza un camino más costoso.



Segmentar los cuellos de botella

Aislar/monitorear **FW-VPN** y **APP-02** rompe el 80% de las rutas de menor resistencia.



Priorizar el eslabón más barato

La arista de menor peso (CVE más crítico, **Log4Shell w=0.1**) es el primer parche.